1. Scenario and requirements

Implement a Student class that decides whether a student has passed an exam. Scores from 0 to 100 are valid; scores of 50 or higher pass. Invalid scores must throw ArgumentException.

2. Test cases

ID	Input score	Expected result
TC-01	75	Pass (true)
TC-02	30	Fail (false)
TC-03	50	Pass (true)
TC-04	49	Fail (false)
TC-05	-10	ArgumentException
TC-06	110	ArgumentException
TC-07	0	Fail
TC – 08	100	Pass
TC-09	51	Pass
TC-10	101	Exception
TC-11	-1	Exception
TC-12	Name=Peter	Name equals Peter
TC-13	Student object created	Object is not null
TC-14	Student object created	Instance of Student
TC-15	Score changes from 40 to 60	Pass
TC-16	Multiple scores	Correct Pass/Fail result

3. Example manual test case: TC-03

Precondition: Student class is implemented.

Steps: 1. Create Student("Henrik", 50). 2. Call HasPassed(). 3. Compare the result.

Expected result: true

Actual result: Record after execution.

Status: Not run / Pass / Fail

4. Codebase/Student.cs

```cs
using System;
namespace Codebase
{
    public class Student
    {
        public string Name { get; set; }
        public int Score { get; set; }

        public Student(string name, int score)
        {
            if (score < 0 || score > 100) throw new ArgumentException("Score must be between 0 and 100");
            Name = name;
            Score = score;
        }

        public bool HasPassed()
        {
            return Score >= 50;
        }
    }
}

```

Program .cs

```cs
using System;

namespace Codebase
{
    public class Program
    {
        public static void Main()
        {
            // Create Student objects
            Student student1 = new Student("Peter", 75);
            Student student2 = new Student("Sara", 30);
            Student student3 = new Student("Henrik", 50);

            // Call HasPassed() method
            Console.WriteLine(student1.Name + ": " + student1.HasPassed());

            Console.WriteLine(student2.Name + ": " + student2.HasPassed());

            Console.WriteLine(student3.Name + ": " + student3.HasPassed());
        }
    }
}
```

5. Create and run the projects in Zed

Open Zed's terminal in a new folder and run the following commands (requires the .NET SDK):

```
dotnet new sln -n StudentTesting
dotnet new console -n Codebase
dotnet new nunit -n UnitTests
dotnet sln add Codebase/Codebase.csproj UnitTests/UnitTests.csproj
dotnet add UnitTests/UnitTests.csproj reference Codebase/Codebase.csproj
```

Create Student.cs in Codebase and StudentTests.cs in UnitTests. Remove the generated sample test file to keep the output clear. Run:

```
dotnet test UnitTests/UnitTests.csproj
```

Expected outcome: 6 passed, 0 failed, if the projects are configured and the code is copied correctly.